Formation 4 :

Encoder une base de données : que faire des Nan

Plan :

Rappels

Comment encoder les Nan

Quelques astuces supplémentaires

Rappels

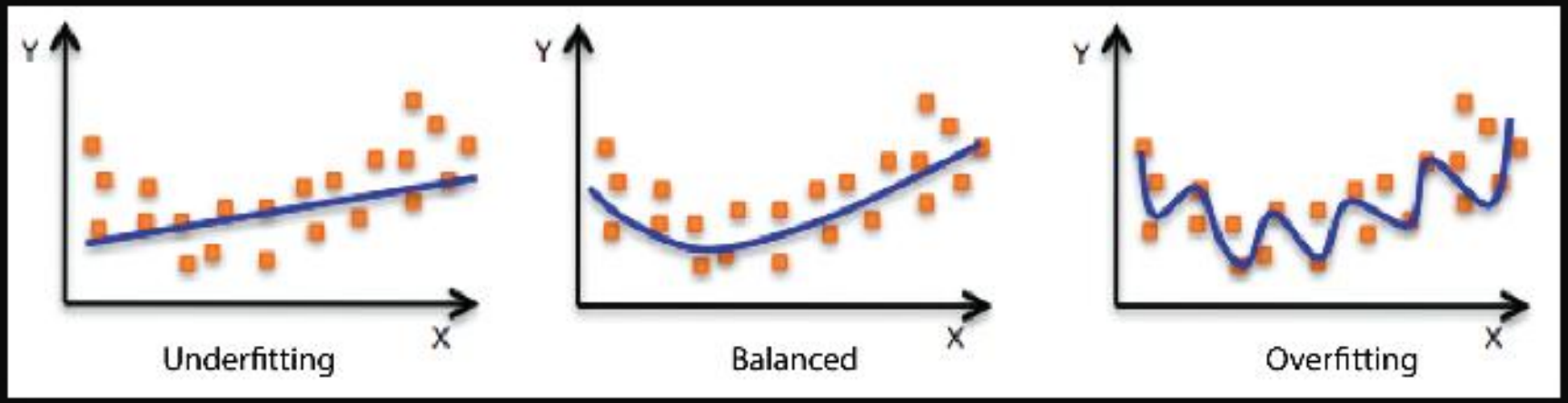
- df = pd.read_csv(« »)

- y = df[‘Cible’]
- X = df.drop (columns = ‘Cible’)

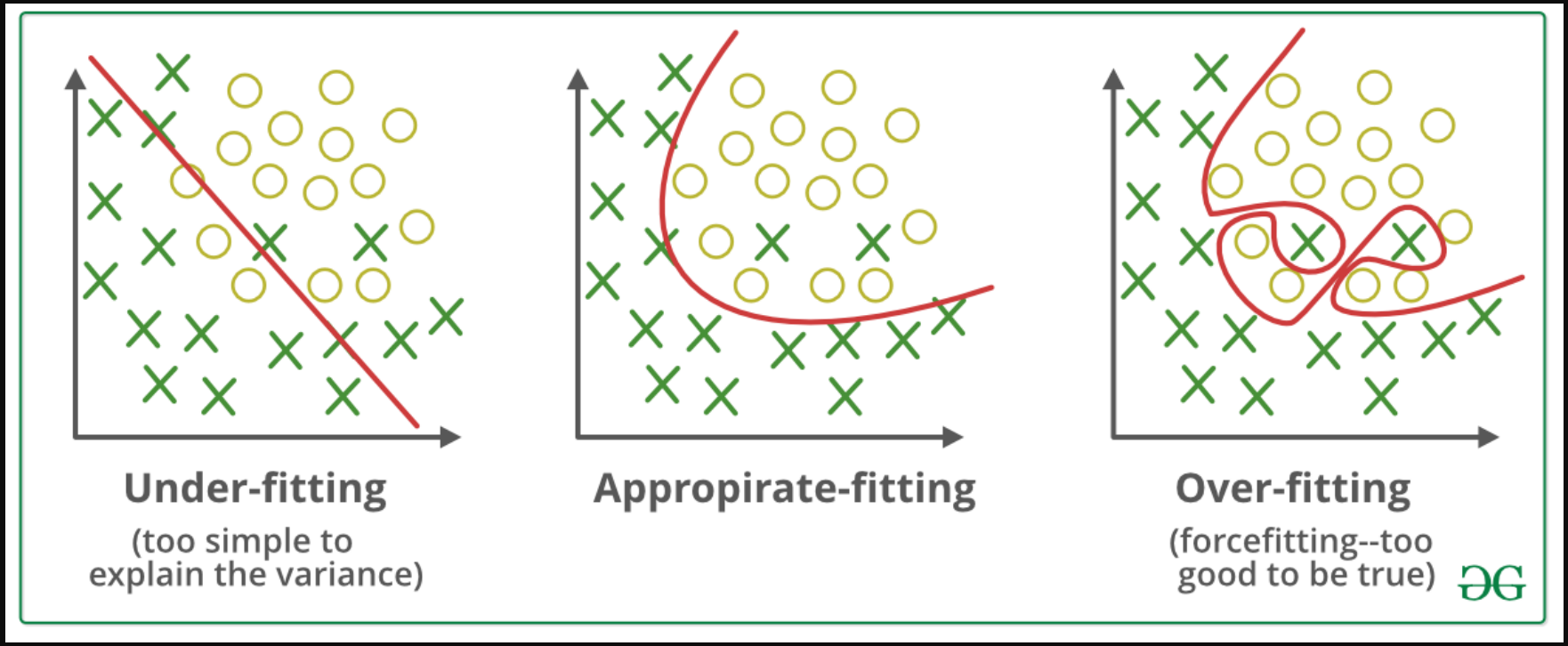
- X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,random_state=42)

- model = xgb.XGBRegressor()
- model.fit(X_train, y_train)

- y_pred = model.predict(X_test)
- rmse = np.sqrt(mean_squared_error(y_test, y_pred))



Underfitting et Overfitting



L’encodage

- C’est l’étape la plus importante ! (et celle qui demande le plus de travail)
- On ne peut pas donner autre chose que des nombres à un modèle
- Attention aux valeurs manquantes

```python
from sklearn.impute import SimpleImputer
my_imputer = SimpleImputer(strategy='mean')
df['column name'] = my_imputer.fit_transform(df[['column name']])
```

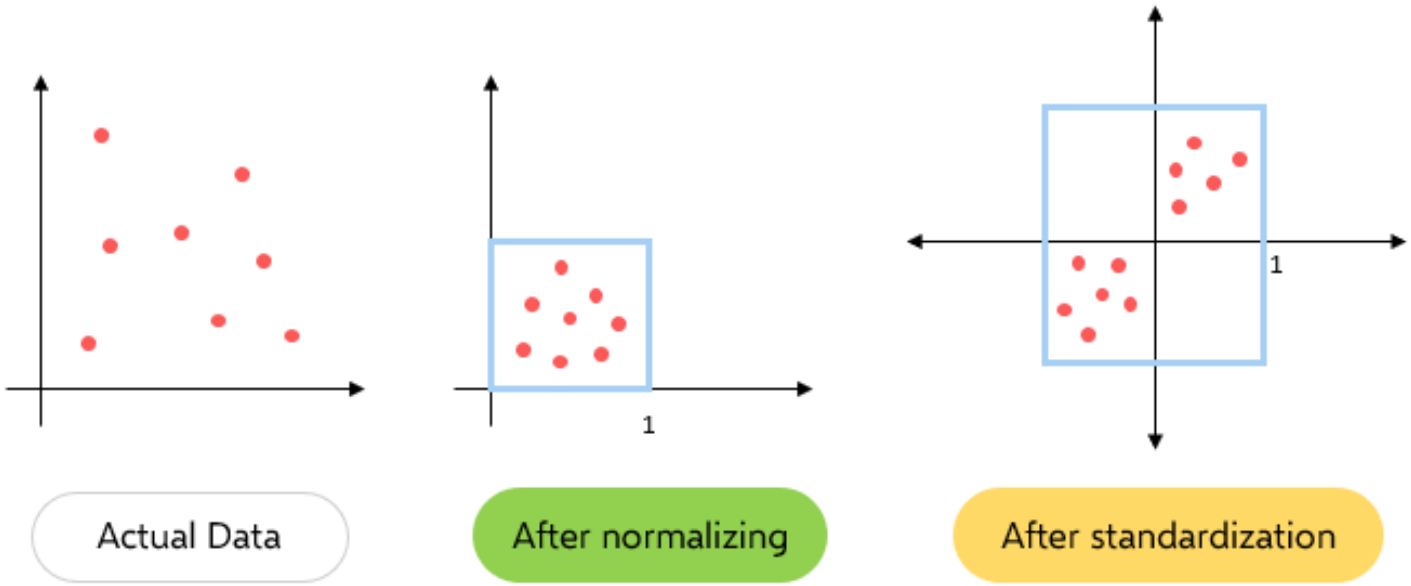
Que faire avec les NaN

- Utiliser un imputer :
- On peut aussi drop les colonnes / lignes avec trop de NaN
- On peut remplacer les NaN par une valeur correspondante

Quelques techniques supplémentaires

$$z = \frac{x - \bar{x}}{\sigma}$$

$$x' = \frac{x - \min(x)}{\max(x) - min(x)}$$

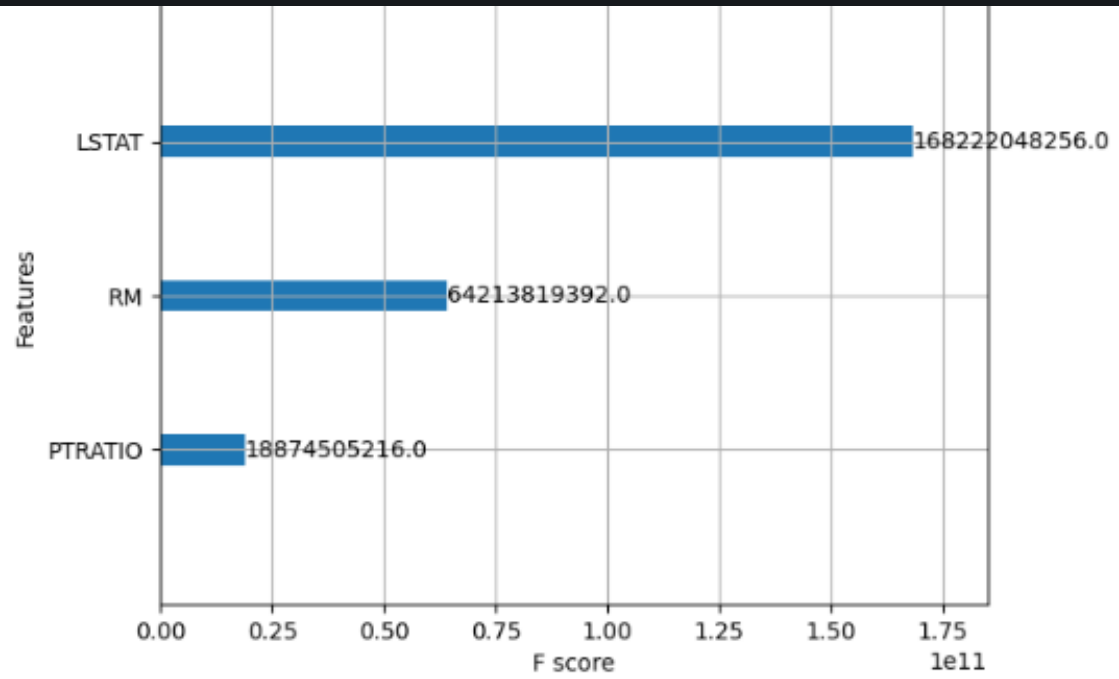


Standardization et Normalization

plot_importance

```
booster = model.get_booster()
booster.feature_names = list(X.columns)

ax = xgb.plot_importance(booster, importance_type="gain", max_num_features=20, show_values=True)
plt.show()
```



A vous de jouer !

TP : Houses Price – Partie 1

- Encoder la BDD puis entrainer dessus un modèle XGboost

- Premières étapes :

1. Aller sur https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques

2. df = pd.read_csv("/kaggle/input/house-prices-advanced-regression-techniques/train.csv")

Ce que vous devez faire

- Drop les colonnes de la BDD avec trop de Nan
- Slectionner uniquement les colonnes numériques de la BDD
- Imputer les Nan dans ces colonnes numériques en les remplaçants par la valeur moyenne de la colonne

- Verifier avec df.info() que toutes les colonnes sont numériques (type float ou int) et ne contiennent pas de Nan

Quelques tips :

- from pandas.api.types import is_integer_dtype, is_float_dtype, is_numeric_dtype

-> Fonctions qui permettent de tester le type d’une colonne de df

- df[c].isna().any() -> teste s’il y a des Nan dans la colonne

- df.columns -> liste de toutes les colonnes de la dataset